

# Informe del módulo de búsqueda web

CSDI RAG Engine

## 1 Módulo de búsqueda web

El módulo de búsqueda web es el mecanismo que permite al sistema responder consultas sobre información que no está presente en el corpus indexado. Su responsabilidad no es sustituir la recuperación local, sino complementarla cuando el corpus no ofrece suficiente evidencia para responder la pregunta del usuario. El módulo se activa automáticamente, de forma transparente para el usuario, como parte del pipeline RAG.

Para que esta activación sea selectiva y no sistemática, el módulo incorpora un componente dedicado a medir la calidad de los resultados locales antes de recurrir a la web. Solo cuando esa calidad cae por debajo de un umbral configurable el sistema ejecuta la búsqueda externa. Esto preserva la rapidez del pipeline en el caso común, donde el corpus contiene la información necesaria, y reserva el costo de la búsqueda web para los casos en que realmente añade valor.

**Arquitectura del módulo.** El módulo de búsqueda web se organiza en cuatro componentes con responsabilidades distintas: el *detector de insuficiencia*, que decide si la evidencia local es suficiente; el *orquestador de búsqueda web*, que coordina la búsqueda externa cuando el detector lo requiere; el *proveedor de búsqueda*, que ejecuta la consulta contra un motor externo; y el *descargador de documentos*, que obtiene y limpia el contenido de las páginas encontradas. Un quinto componente transversal es el *índice de caché web*, que persiste los documentos descargados para que puedan ser recuperados en consultas futuras sin repetir la búsqueda externa.

Estos componentes no operan en paralelo: forman una cadena activada condicionalmente dentro del pipeline RAG. La cadena completa solo se ejecuta cuando el corpus local no satisface la consulta, y dentro de ella se intenta primero el caché antes de invocar el motor externo.

**Diseño general del flujo de búsqueda web.** Cuando el pipeline RAG recupera fragmentos del corpus y los entrega al detector de insuficiencia, el detector calcula una puntuación de confianza local. Si esa puntuación supera el umbral configurado, el pipeline procede directamente a la generación de respuesta. Si no lo supera, el pipeline consulta primero el caché web: fragmentos de búsquedas anteriores ya indexados. Si el caché tampoco es suficiente, se ejecuta la búsqueda externa, se descargan y chunkean los documentos encontrados, se indexan en el caché web y se recuperan inmediatamente para la consulta actual. En cualquier punto donde la suficiencia se recupere, la cadena se detiene.

## 2 Detector de insuficiencia

El detector de insuficiencia (`InsufficiencyDetector`) es el componente que determina si los fragmentos recuperados del corpus local son suficientes para responder la consulta. Esta decisión no se

toma con un umbral simple sobre el número de resultados ni sobre la puntuación máxima: se calcula una puntuación de confianza compuesta a partir de cinco señales independientes que capturan aspectos distintos de la calidad de la recuperación.

**Métricas calculadas.** El detector calcula nueve valores sobre el conjunto de fragmentos recuperados, de los cuales cinco contribuyen directamente a la puntuación de confianza final:

- **top\_score\_norm**: puntuación máxima de los fragmentos, normalizada respecto al máximo teórico del método de fusión. Cuando la fusión es RRF con constante  $k$  y suma de pesos  $W$ , el máximo teórico es  $W/(k+1)$ ; con  $k = 60$  y  $W = 1,0$  ese máximo es  $1/61 \approx 0,0164$ . Normalizar permite comparar el score de forma independiente a la escala de los recuperadores.
- **quantity\_score**: proporción de fragmentos recuperados respecto a la cantidad esperada, calculada como  $\min(1,0, n_{\text{results}}/n_{\text{expected}})$  con  $n_{\text{expected}} = 10$ . Satura en 1 cuando hay al menos diez fragmentos.
- **coverage\_score**: fracción promedio de términos de la consulta presentes en los  $N$  fragmentos de mayor puntuación (por defecto  $N = 5$ ). Para cada fragmento se calcula el cociente entre los términos de la consulta que aparecen en su texto y el total de términos de la consulta; el coverage\_score es la media de esos cocientes. Mide si los fragmentos recuperados contienen el vocabulario de la pregunta.
- **diversity\_score**: cociente entre el número de URLs únicas y el número total de fragmentos. Un valor bajo indica que los fragmentos provienen de una sola fuente, lo que reduce la robustez de la respuesta frente a errores o sesgos de esa fuente.
- **answerability\_score**: estimación de la capacidad de respuesta combinando dos señales: la proporción de fragmentos con cobertura suficiente de la consulta (*relevant\_ratio*) y el coverage\_score general. La fórmula es:

$$\text{answerability} = \text{clamp}_{[0,1]} \left( 0,6 \cdot \frac{\min(r, r_{\min})}{r_{\min}} + 0,4 \cdot \text{coverage\_score} \right)$$

donde  $r$  es el número de fragmentos cuyo overlap con la consulta supera el umbral  $\theta_{\text{overlap}} = 0,20$  y  $r_{\min} = 2$  es el mínimo esperado. El coeficiente 0,6 pondera la profundidad (¿hay al menos dos fragmentos que hablan del tema?) y el 0,4 la amplitud (¿cubren el vocabulario de la consulta?).

**Tokenización para cobertura y answerability.** El cálculo de cobertura requiere comparar los términos de la consulta con los del texto de cada fragmento. El tokenizador usado, `simple_tokenize`, aplica expresiones regulares Unicode (`[^\W_]+`) para extraer palabras en minúscula, filtra stopwords en inglés y en español, y descarta tokens de un solo carácter. El soporte explícito de stopwords en español garantiza que la cobertura no se infle por palabras vacías en consultas formuladas en ese idioma.

**Fórmula de confianza local.** Las cinco métricas se combinan en una puntuación de confianza ponderada:

$$c_{\text{local}} = \text{clamp}_{[0,1]} (w_{\text{top}} \cdot \text{top\_score\_norm} + w_q \cdot \text{quantity\_score} + w_c \cdot \text{coverage\_score} + w_d \cdot \text{diversity\_score} + w_a)$$

con valores por defecto  $w_{\text{top}} = 0,10$ ,  $w_q = 0,15$ ,  $w_c = 0,35$ ,  $w_d = 0,15$ ,  $w_a = 0,25$ . Los pesos suman exactamente 1,0; el sistema valida esta invariante al arrancar y rechaza configuraciones inválidas.

**Justificación de los pesos.** La distribución de pesos no es uniforme porque las métricas no tienen el mismo poder predictivo sobre la calidad de la recuperación. El `coverage_score` recibe el mayor peso (0,35) porque es el mejor proxy disponible de relevancia léxica sin necesitar un modelo adicional: si los fragmentos recuperados no contienen los términos de la consulta, es improbable que puedan responderla. El `answerability_score` recibe el segundo peso más alto (0,25) porque combina dos señales y captura tanto la profundidad como la anchura de la cobertura. El `top_score_norm` recibe el peso más bajo (0,10) deliberadamente: una puntuación alta de fusión indica que el recuperador híbrido encontró buenos candidatos según sus propias métricas de similitud, pero no garantiza que esos fragmentos contengan la respuesta a la pregunta específica del usuario.

**Decisión de búsqueda web y razones diagnósticas.** La decisión de activar la búsqueda web se toma comparando  $c_{\text{local}}$  con el umbral  $\theta_c = 0,65$ : si  $c_{\text{local}} < \theta_c$ , el sistema requiere búsqueda web. De forma independiente a ese umbral, el detector evalúa condiciones individuales sobre cada métrica y construye una lista de razones diagnósticas. Las razones posibles son: `NO_RESULTS` (sin fragmentos), `LOW_NUM_RESULTS` (menos de 5 fragmentos), `LOW_TOP_SCORE` (`top_score_norm` < 0,35), `LOW_COVERAGE` (`coverage` < 0,20), `LOW_SOURCE_DIVERSITY` (`diversity` < 0,30), `LOW_ANSWERABILITY` (`answerability` < 0,40) y `LOW_CONFIDENCE` (confianza global bajo el umbral). Esta lista de razones se devuelve junto con la decisión y permite auditar por qué se activó la búsqueda en cada consulta, lo que facilita el diagnóstico y el ajuste de parámetros.

### 3 Caché web

Antes de ejecutar una búsqueda externa, el pipeline consulta el caché web: un índice híbrido independiente del corpus principal que almacena los fragmentos obtenidos en búsquedas web anteriores. La separación física entre el corpus principal y el caché web es una decisión de diseño deliberada: garantiza que los documentos descargados de la web no contaminen el índice del corpus curado y que puedan gestionarse con políticas distintas de retención y actualización.

**Estructura del caché web.** El caché web replica la arquitectura de índice del corpus principal pero en tablas separadas: `web_cache_chunks` para los fragmentos de texto, `web_cache_bm25_*` para el índice invertido léxico y `web_cache_vector_*` para el índice FAISS de embeddings. La misma clase `HybridRetriever` que opera sobre el corpus puede operar sobre el caché web con distintas instancias de repositorios, reutilizando toda la lógica de fusión RRF y reordenamiento sin duplicar código.

Los documentos completos descargados se persisten en `web_cache_documents` con deduplicación por URL mediante un `INSERT ... ON CONFLICT (url) DO UPDATE`. Esto evita indexar el mismo documento dos veces si la misma URL aparece en búsquedas distintas.

**Evaluación del caché antes de la búsqueda externa.** Cuando el detector de insuficiencia indica que el corpus no es suficiente, el pipeline ejecuta una recuperación sobre el caché web y somete los resultados al mismo detector. Si los fragmentos del caché elevan la confianza por encima del umbral, la búsqueda externa no se ejecuta. Este mecanismo de doble evaluación convierte el caché en un amortiguador eficiente: búsquedas repetidas sobre temas similares reutilizan los documentos ya descargados sin incurrir en el costo de una nueva llamada al motor externo.

## 4 Orquestador de búsqueda web

El `WebSearchOrchestrator` es el componente que coordina la búsqueda externa cuando ni el corpus ni el caché son suficientes. Recibe la consulta, la pasa al proveedor de búsqueda, descarga los documentos encontrados, los fragmenta, los indexa en el caché web y persiste un registro de la búsqueda. Todo esto ocurre dentro de una sola llamada a `run(query)`, que devuelve un `WebSearchRunResult` con los hits obtenidos, los documentos descargados y el número de fragmentos indexados.

Si la búsqueda está deshabilitada (`WEB_SEARCH_ENABLED=false`) o la consulta está vacía, el orquestador devuelve un resultado vacío sin ejecutar ninguna operación. Este punto de desactivación explícita permite operar el sistema en modo *corpus-only* durante pruebas o en entornos sin acceso a internet.

**Persistencia de búsquedas.** Cada ejecución del orquestador se registra en la tabla `web_search_runs` con la consulta normalizada, el proveedor utilizado, el número de hits obtenidos y el número de fragmentos indexados. Este registro permite auditar la actividad de búsqueda externa, detectar patrones de insuficiencia recurrentes en el corpus y evaluar la efectividad del caché a lo largo del tiempo.

## 5 Proveedor de búsqueda: DuckDuckGo

El proveedor de búsqueda implementado es `DuckDuckGoSearchProvider`, que consulta la interfaz HTML de DuckDuckGo sin requerir clave de API. La búsqueda se realiza mediante una petición HTTP GET al endpoint `https://duckduckgo.com/html/` con el parámetro `q=<consulta>`, y los resultados se extraen del HTML devuelto mediante selectores CSS con BeautifulSoup.

**Normalización de enlaces.** DuckDuckGo encapsula las URLs de los resultados en redirecciones internas con el formato `/1/?uddg=<url_codificada>`. El proveedor detecta este patrón y decodifica la URL real antes de devolver el hit, evitando que el descargador intente acceder a la URL de redirección en lugar del documento original.

**Justificación de la elección del proveedor.** DuckDuckGo fue seleccionado porque no requiere registro ni clave de API para uso programático de su interfaz HTML, lo que elimina una dependencia de configuración externa para el funcionamiento básico del sistema. La contrapartida es que la interfaz HTML no es una API oficial y puede cambiar sin aviso, lo cual se identificó explícitamente como una limitación del diseño (véase sección de limitaciones). Para entornos de producción con mayor estabilidad, el diseño de la interfaz `SearchProvider` como `Protocol` permite sustituir el proveedor por cualquier implementación alternativa sin modificar el resto del módulo.

## 6 Descargador de documentos

El `HttpDocumentFetcher` descarga el contenido de cada URL retornada por el proveedor. Para cada hit, realiza una petición HTTP GET con un *User-Agent* identificable, verifica el tipo de contenido de la respuesta y selecciona la estrategia de extracción de texto apropiada.

**Extracción de contenido HTML.** Para respuestas con `Content-Type: text/html`, el descargador delega en el `Scraper` del sistema, que aplica selectores configurados para extraer el contenido principal de la página: se seleccionan elementos `main`, `article` o `[role=main]`, excluyendo `script`,

`style` y `noscript`. El título se extrae del primer `h1` o del elemento `title`. Para respuestas no HTML, se usa el texto plano de la respuesta directamente.

Tras la extracción, el texto se normaliza colapsando secuencias de espacios y saltos de línea en un único espacio, y se trunca a un máximo de 10 000 caracteres. Este límite evita que páginas muy largas generen un número excesivo de fragmentos y degraden el rendimiento del índice. Si el texto resultante está vacío, el descargador devuelve `None` y el orquestador omite ese documento. Los fallos de red o HTTP se registran en el log y no interrumpen la descarga de los demás documentos del mismo lote.

**Fragmentación e indexación.** Los documentos descargados se fragmentan con el mismo `Chunker` usado para el corpus principal (256 palabras, solapamiento de 32 palabras). Cada fragmento recibe un `source_id` con el formato `web:<proveedor>:<dominio>`, por ejemplo `web:duckduckgo:docs.python.org`, y un campo `breadcrumb="web-search"`. Este campo permite que el pipeline identifique de forma unívoca qué fragmentos de la respuesta final provienen de búsquedas web frente a los que provienen del corpus curado.

Los fragmentos se ingresan en el caché web mediante el mismo servicio de ingesta del corpus, que actualiza tanto el índice BM25 como el vectorial. Tras la ingesta, el índice BM25 del caché se reconstruye para reflejar los nuevos documentos antes de que el pipeline realice la recuperación post-búsqueda.

## 7 Integración en el pipeline RAG

El módulo de búsqueda web no opera de forma independiente: es una rama condicional del `RAGPipeline`. La decisión de activarla, el uso del caché y la recuperación post-búsqueda están coordinados por el mismo pipeline que gestiona la recuperación del corpus y la generación de respuesta.

**Flujo condicional completo.** El pipeline evalúa la suficiencia en dos momentos: después de la recuperación del corpus y después de la recuperación del caché web. El flujo completo es:

1. Recuperación híbrida sobre el corpus principal.
2. Evaluación de suficiencia con `InsufficiencyDetector`.
3. Si insuficiente y caché habilitado: recuperación sobre el caché web y segunda evaluación.
4. Si aún insuficiente y búsqueda externa habilitada: ejecución del orquestador, indexación de documentos nuevos y recuperación sobre el caché actualizado.
5. Reordenamiento con cross-encoder sobre el conjunto final de fragmentos.
6. Generación de respuesta con el LLM.

En cualquier punto donde la suficiencia se recupere, los pasos siguientes no se ejecutan. Si el `WebSearchOrchestrator` no está configurado (instancia `None`), el pipeline responde con los fragmentos disponibles del corpus sin intentar la búsqueda externa, lo que permite operar el sistema en modo degradado sin errores.

**Trazabilidad de fuentes en la respuesta.** El resultado devuelto al cliente incluye una lista de fuentes con el campo `source_type`, que distingue entre `"corpus"` y `"web_cache"`. Esta distinción permite al usuario saber si la información proviene del corpus curado del sistema o de documentos obtenidos en tiempo real de la web, lo que es relevante para evaluar la fiabilidad de la respuesta. Adicionalmente, los campos `external_search_executed` y `external_indexed_count` del resultado informan si se ejecutó una búsqueda externa y cuántos fragmentos se indexaron.

## 8 Configuración y parámetros clave

El módulo de búsqueda web expone 23 parámetros configurables mediante variables de entorno, organizados en grupos funcionales. Los más relevantes para el comportamiento del sistema son:

| Variable                       | Default | Efecto                                      |
|--------------------------------|---------|---------------------------------------------|
| WEB_SEARCH_ENABLED             | true    | Activa o desactiva el módulo completo       |
| WEB_SEARCH_TOP_K               | 5       | Resultados solicitados al proveedor         |
| WEB_CACHE_ENABLED              | true    | Consulta el caché antes de búsqueda externa |
| WEB_CACHE_TOP_K                | 10      | Fragmentos recuperados del caché            |
| INSUFF_CONFIDENCE_THRESHOLD    | 0.65    | Umbral global de confianza                  |
| INSUFF_W_COVERAGE              | 0.35    | Peso del coverage en la confianza           |
| INSUFF_W_ANSWERABILITY         | 0.25    | Peso del answerability en la confianza      |
| INSUFF_W_TOP                   | 0.10    | Peso del top_score en la confianza          |
| INSUFF_MIN_COVERAGE_SCORE      | 0.20    | Mínimo de coverage para no marcar razón     |
| INSUFF_MIN_ANSWERABILITY_SCORE | 0.40    | Mínimo de answerability                     |
| INSUFF_COVERAGE_TOP_N          | 5       | Fragmentos evaluados para coverage          |

La suma de los cinco pesos del detector es validada al cargar la configuración: si no suman exactamente 1,0 (con tolerancia  $10^{-6}$ ), el sistema rechaza la configuración con un error explícito. Esto evita que configuraciones incorrectas produzcan puntuaciones de confianza fuera del rango  $[0, 1]$  de forma silenciosa.

## 9 Justificación técnica de la solución

El módulo de búsqueda web existe porque ningún corpus estático puede anticipar todas las consultas posibles de los usuarios. Documentación desactualizada, temas de larga cola no cubiertos por el corpus, o preguntas sobre eventos recientes son casos que la recuperación local no puede resolver por diseño. Sin un mecanismo de extensión, el sistema generaría respuestas incorrectas o reconocería su ignorancia en situaciones donde la información existe y es accesible.

La decisión de no activar la búsqueda web en cada consulta responde a un balance entre calidad y costo. Una búsqueda externa en cada consulta añadiría latencia constante, dependencia de un servicio externo y ruido potencial de documentos no relevantes, sin beneficio cuando el corpus ya contiene la respuesta. El detector de insuficiencia resuelve este balance estimando la calidad local antes de incurrir en ese costo.

**Por qué una puntuación compuesta y no un umbral simple.** Un umbral simple sobre el número de resultados o sobre la puntuación máxima del recuperador sería insuficiente porque cada señal captura solo una dimensión del problema. Una consulta puede tener muchos resultados con puntuaciones altas pero todos provenientes de la misma fuente y sin cubrir los términos de la

pregunta: el número y el score serían altos pero la respuesta sería de baja calidad. La puntuación compuesta detecta este caso porque el `coverage_score` y el `diversity_score` serían bajos, arrastrando la confianza por debajo del umbral.

**Por qué el caché web es un componente separado y no parte del corpus.** Los documentos del corpus son curados y estructurados; los documentos de la web son heterogéneos, potencialmente ruidosos y temporalmente válidos. Mezclarlos en el mismo índice degradaría la calidad del corpus y complicaría las operaciones de mantenimiento como la reconstrucción del índice o la actualización de documentos. La separación permite aplicar políticas distintas a cada índice y garantiza que el corpus principal no se vea afectado por el volumen o calidad variable de los documentos web.

**Por qué se reutiliza la arquitectura de recuperación del corpus.** El caché web usa las mismas clases `HybridRetriever`, `BM25Retriever` y `VectorRetriever` que el corpus, instanciadas con repositorios distintos. Esta decisión evita duplicar la lógica de recuperación y garantiza que el caché se beneficie de las mismas mejoras que se apliquen al corpus: si se ajustan los pesos de fusión o se cambia el modelo de embeddings, el caché hereda esos cambios sin intervención adicional.

## 10 Limitaciones del módulo

El módulo de búsqueda web tiene limitaciones conocidas que acotan su alcance y confiabilidad.

El proveedor DuckDuckGo opera sobre la interfaz HTML pública, que no es una API oficial. Esta interfaz puede cambiar su estructura sin aviso previo, lo que puede romper el parser de resultados. En ese caso, el módulo devolvería cero resultados sin error explícito hasta que el extractor sea actualizado.

La descarga de documentos falla silenciosamente para URLs que requieren autenticación, que devuelven contenido dinámico generado por JavaScript, o que bloquean el *User-Agent* del sistema. En todos esos casos el documento es omitido, lo que puede reducir el número efectivo de fragmentos disponibles tras la búsqueda.

El truncado de documentos a 10 000 caracteres puede cortar información relevante en páginas largas. Para documentación técnica extensa, como páginas de referencia de API o tutoriales completos, los fragmentos finales del documento pueden quedar fuera del límite y no ser indexados.

El caché web no tiene política de expiración implementada. Documentos indexados en búsquedas anteriores permanecen en el caché indefinidamente, lo que puede provocar que consultas posteriores recuperen información desactualizada sin que el sistema lo detecte. Este problema es especialmente relevante para consultas sobre información que cambia con el tiempo.

La evaluación de suficiencia se basa en señales léxicas (`coverage`) y de cantidad, no en una estimación semántica profunda de si los fragmentos contienen la respuesta correcta. Es posible que el detector no active la búsqueda web en casos donde los fragmentos recuperados son temáticamente relevantes pero factualmente incorrectos o incompletos.